Лекция 2
Lisp и основы Clojure



01

Концепции функционального программирования

Чистые функции

Чистая функция для одинаковых аргументов возвращает одинаковый результат и не имеет наблюдаемых побочных эффектов.

```
(defn square [x]
  (* x x))

(square 5) ;; => 25
(square 5) ;; => 25
```

Примеры побочных эффектов

Побочный эффект — наблюдаемое взаимодействие вычисления с состоянием или внешним миром.

- изменение разделяемого или глобального состояния
- ввод и вывод: консоль, файлы, логирование
- сетевые запросы и обращения к базе данных
- чтение времени, случайных значений, переменных окружения
- генерация исключения
- запуск или взаимодействие с другими процессами / потоками

Ссылочная прозрачность

Выражение ссылочно прозрачно, если его можно заменить его значением без изменения поведения программы.

```
(+ (square 5) (square 5))
```

⇓

```
(+ 25 25)
```

Неизменяемые данные

Неизменяемое значение после создания не меняется: операция создает новое значение.

```
(def xs '(1 2 3))
(def ys (cons 0 xs))

xs ;; => (1 2 3)
ys ;; => (0 1 2 3)
```

Функции как объекты первого класса

Функция — обычное значение: её можно связать с именем, передать как аргумент и вернуть как результат.

```
(def operation +)
(operation 2 3) ;; => 5
(defn apply-twice [f x]
  (f (f x)))

(apply-twice inc 10) ;; => 12
```

02

Языки семейства Lisp

Почему в Lisp синтаксис почти сразу становится данными?

Reader и evaluator

Reader — компонент, который превращает текст программы в структуры данных языка.

Evaluator — компонент, который вычисляет форму и получает значение.

`текст программы → reader → данные → evaluator → значение`

Что делает reader?

```
текст программы
   ↓ reader
структура данных
   ↓ evaluator
значение
```

Где здесь синтаксис, а где семантика?

S-выражение: определение

S-выражение — рекурсивная форма представления данных, состоящая из атомов и списков S-выражений.

Атом — неделимое с точки зрения S-выражения значение: символ, число, строка, nil и т. п.

```
S ::= atom | (S1 S2 ... Sn)
```

Lisp и S-выражения

Lisp – семейство языков, где программы записываются в виде S-выражений.

```
S ::= atom | (S1 S2 ... Sn)
```

Примеры S-выражений

```
42
true
foo
(+ 1 2)
(* (+ 1 2) 3)
```

Список может быть данными, а может быть формой для вычисления — это решает evaluator.

Аппликация функции

Аппликация функции — применение функции к одному или нескольким аргументам.

```
(+ 1 2)     (max 10 20)     (str "a" "b")
```

Не каждый список — аппликация функции

```
(if true
  42
  (/ 1 0))
```

Если бы if был обычной функцией,
оба аргумента вычислялись бы заранее

Специальная форма

Специальная форма — конструкция, чьи правила вычисления отличаются от обычной аппликации.

`if` `let` `fn` `quote` `def`

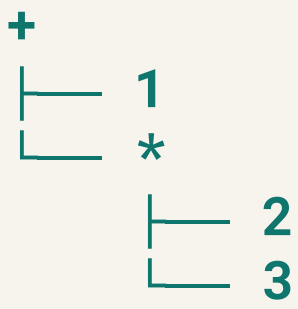
03

Код как данные

Синтаксическое дерево

Синтаксическое дерево — древовидное представление структуры программы.

```
(+ 1 (* 2 3))
```



S-выражение ≠ AST

S-выражение — синтаксическое представление, из которого может быть построено AST.

Гомоиконичность

Гомоиконичность — свойство языка, при котором код имеет форму данных языка.

Почему это важно

В Lisp программа может анализировать и преобразовывать код обычными функциями.

quote

```
(+ 1 2)
;; => 3

'(+ 1 2)
;; => (+ 1 2)
```

quote возвращает
выражение как данные,
не вычисляя его.

'x = (quote x)

quote → eval

```
(def code '(+ 1 2))

code
;; => (+ 1 2)

(eval code)
;; => 3
```

Изменяем код как данные

```
(def code '(+ 1 2))
(def new-code (cons '* (rest code)))

new-code
;; => (* 1 2)

(eval new-code)
;; => 2
```

Обычная операция позволяет менять алгоритм: eval вычисляет уже новую форму!

04

Основные конструкции Clojure

def: связывание имени со значением

def создаёт переменную в текущем namespace и связывает его с вычисленным значением.

```
(def answer 42)
answer
;; => 42

(def numbers '(1 2 3))
numbers
;; => (1 2 3)
```

Функции

```
(fn [x]
  (* x x)) ;; без имени

(defn square [x]
  (* x x)) ;; с именем
```

λx.M ↔ (fn [x] M)

defn связывает имя с функцией.

if: ветвление

if вычисляет условие и ровно одну из двух ветвей.

```
(if (< x 0)
  (- x) ;; ‘-’ тоже функция
  x)
```

cond: множественное ветвление

cond последовательно проверяет условия и вычисляет выражение первой подходящей ветви.

```
(cond
  (< x 0) :negative
  (= x 0) :zero
  :else   :positive)
```

let: локальные связывания

let создает локальные лексические связывания, доступные только внутри его тела.

```
(defn total-price [price quantity]
  (let [subtotal (* price quantity)
        tax (* subtotal 0.2)]
    (+ subtotal tax)))
```

Связывания вычисляются слева направо; последующие могут использовать предыдущие.

Разбор кода: что здесь происходит?

```
(defn classify [x]
  (let [abs-x (if (< x 0) (- x) x)]
    (cond (= abs-x 0) :zero
          (< abs-x 10) :small
          :else :large)))
```

Где let? Где if? Где cond?

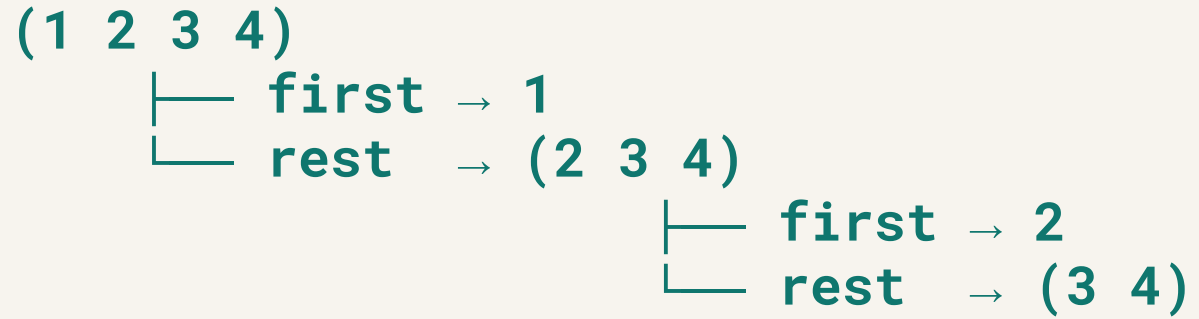
05

Списки и рекурсия

Список как рекурсивная структура

Список

Список — пустой список либо пара: первый элемент + оставшаяся часть списка.



Структурная рекурсия

Структурная рекурсия — рекурсия, в которой рекурсивные вызовы выполняются над непосредственными подструктурами исходных данных

```
пустой список → базовый случай
непустой список → обработать first
                    + рекурсивно обработать rest
```

Обычная рекурсия: дерево вызовов

```
(defn my-sum [xs]
  (if (empty? xs)
    0
    (+ (first xs)
       (my-sum (rest xs)))))
```

Обычная рекурсия: дерево вызовов

```
(my-sum '(1 2 3))

my-sum (1 2 3)
   └─ 1 + my-sum (2 3)
            └─ 2 + my-sum (3)
                     └─ 3 + my-sum ()
                              └─ 0
```

Стек хранит незавершенные операции: 1 +, 2 +, 3 +

Хвостовая рекурсия

Хвостовая рекурсия: рекурсивный вызов находится в хвостовой позиции.

my-sum: хвостовая рекурсия

Данная реализация не оптимизируется интерпретатором. Нужен recur

```
(defn my-sum-impl [xs acc]
  (if (empty? xs)
    acc
    (my-sum-impl (rest xs)
                 (+ acc (first xs)))))

(defn my-sum [xs]
  (my-sum-impl xs 0))
```

my-sum: хвостовая рекурсия

```
(my-sum-impl '(1 2 3) 0)

my-sum-impl (1 2 3), 0
    └─ my-sum-impl (2 3), 0 + 1
           └─ my-sum-impl (3), 1 + 2
                  └─ my-sum-impl (), 6
                         └─ 6
```

Хвостовая рекурсия: recur

```
(defn my-sum-impl [xs acc]
  (if (empty? xs)
    acc
    (recur (rest xs)
           (+ acc (first xs)))))

(defn my-sum [xs]
  (my-sum-impl xs 0))
```

Внутри функции recur передаёт новые аргументы и возвращается к точке рекурсии этой функции — без роста стека.

06

Еще раз функции

Короткая запись анонимной функции: #(...)

#(...) — сокращённая запись анонимной функции. Аргументы обозначаются через %.

```
#(* % %)
;; то же, что (fn [x] (* x x))

#(+ %1 %2)
;; то же, что (fn [x y] (+ x y))
```

Функции с переменным числом аргументов

```
(defn greet [greeting & names]
  (str greeting ", " names))

(greet "Hello" "Alice" "Bob")
;; names => ("Alice" "Bob")
```

В списке параметров & отделяет обязательные параметры от остальных.

Перегрузка функций по числу аргументов

```
(defn greet
  ([name]
   (greet "Hello" name))
  ([greeting name]
   (str greeting ", " name)))

(greet "Alice") ;; вызов первого варианта
(greet "Hi" "Alice") ;; вызов второго
```

my-sum: recur и перегрузка

```
(defn my-sum
  ([xs]
   (my-sum xs 0))
  ([xs acc]
   (if (empty? xs)
     acc
     (recur (rest xs)
            (+ acc (first xs))))))
```

[xs] — внешний интерфейс; [xs acc] — рекурсивный вариант с аккумулятором.

07

Коллекции

Основные коллекции

```
list:   '(1 2 3)
vector:  [1 2 3]
set:    #{1 2 3}
map:     {:name "Alice" :age 25}
```

Все эти структуры неизменяемы: операции создают новые значения.

Основные операции со списками

```
(list 1 2 3)              ;; => (1 2 3)
(first '(1 2 3))          ;; => 1
(rest '(1 2 3))           ;; => (2 3)
(cons 0 '(1 2 3))         ;; => (0 1 2 3)
(empty? '())              ;; => true
(nth '(1 2 3 4 5) 3)      ;; => 4
```

first и rest разбирают список;
cons строит новый список, добавляя элемент в начало.

Основные операции с векторами

```
(vector 1 2 3)          ;; => [1 2 3]
(nth [1 2 3] 1)         ;; => 2
(conj [1 2 3] 4)        ;; => [1 2 3 4]
(assoc [1 2 3] 1 10)    ;; => [1 10 3]
(pop [1 2 3])           ;; => [1 2]
```

Вектор поддерживает эффективный доступ по индексу; операции возвращают новый вектор, не изменяя исходный.

Основные операции с map

```
(def user {:name "Alice" :age 25})

(get user :name)           ;; => "Alice"
(:age user)                ;; => 25
(assoc user :age 26)
            ;; => {:name    "Alice", :age 26}
(dissoc user :age)
            ;; => {:name "Alice"}
(contains? user :name)     ;; => true
```

Map хранит соответствия ключ → значение;
assoc и dissoc возвращают новую map.

Destructuring последовательностей

Destructuring позволяет связать части структуры данных с локальными именами.

```
(let [[x y] [10 20]]
  (+ x y))
;; => 30

(let [[x & xs] [1 2 3 4]]
  [x xs])
;; => [1 (2 3 4)]
```

Destructuring map

Для map можно извлекать значения по ключам прямо при связывании.

```
(def user {:name "Alice" :age 25})

(let [{:keys [name age]} user]
  (str name " is " age))
;; => "Alice is 25"
```

:keys создает локальные имена из одноименных keyword-ключей.

08

Обработка данных

map

map применяет функцию к каждому элементу последовательности.

```
(map inc [1 2 3])
;; => (2 3 4)

(map (fn [x] (* x x)) [1 2 3])
;; => (1 4 9)
```

filter

filter оставляет элементы, для которых предикат возвращает truthy-значение.

```
(filter even? [1 2 3 4 5 6])
;; => (2 4 6), оставляем только четные

(filter pos? [-2 -1 0 1 2])
;; => (1 2), оставляем положительные
```

reduce

reduce последовательно объединяет элементы, накапливая один результат.

```
(reduce + 0 [1 2 3 4])
;; => 10

(reduce * 1 [1 2 3 4])
;; => 24
```

my-sum через reduce

Суммирование списка — частный случай свертки последовательности.

```
(defn my-sum [xs]
  (reduce + 0 xs))

(my-sum [1 2 3 4])
;; => 10
```

Аккумулятор и рекурсивный обход теперь скрыты внутри reduce.

09

Мемоизация

Мемоизация

Мемоизация — запоминание результатов вызовов функции для повторного использования при тех же аргументах.

```
(defn slow-square [x]
  (Thread/sleep 1000)
  (* x x))

(def fast-square (memoize slow-square))

(fast-square 10) ;; вычисление
(fast-square 10) ;; сохраненный результат
```

Мемоизация и побочные эффекты

Сколько раз будет напечатано Computing...?

```
(defn noisy-square [x]
  (println "Computing...")
  (* x x))

(def mem-square (memoize noisy-square))

(mem-square 10)
(mem-square 10)
```

Мемоизация — не серебряная пуля

Кеш полезен только тогда, когда повторное вычисление действительно дороже хранения результата.

- Кеш потребляет память.
- Если аргументы почти не повторяются, попаданий в кэш мало.
- Дешёвую функцию иногда быстрее вычислить заново.
- Побочные эффекты и зависимость от внешнего состояния делают кеширование опасным.

10

Java Interop

Java Interop

Clojure работает на JVM и позволяет напрямую вызывать Java-классы и методы.

```
;; статический метод
(Math/sqrt 25)        ;; => 5.0

;; конструктор
(String. "hello")

;; метод экземпляра
(.toUpperCase "hello") ;; => "HELLO"
```

Java Interop: пример

Java-объект можно создать и использовать из обычного Clojure-кода.

```
(def now (java.time.LocalDate/now))

(.getYear now)
;; => 2026

(.plusDays now 7)
;; => дата через 7 дней
```